CHAPTER 10

NEIGHBORHOOD PROCESSING

WHAT WILL WE LEARN?

- What is neighborhood processing and how does it differ from point processing?
- What is convolution and how is it used to process digital images?
- What is a low-pass linear filter, what is it used for, and howcan it be implemented using 2D convolution?
- What is a median filter and what is it used for?
- What is a high-pass linear filter, what is it used for, and howcan it be implemented using 2D convolution?

NEIGHBORHOOD PROCESSING

- Main steps:
  - Define a reference point in the input image, $f(x0, y0)$.
  - Perform an operation that involves only pixels within a neighborhood around the reference point in the input image.
  - Apply the result of that operation to the pixel of same coordinates in the output image, $g(x0, y0)$.
  - Repeat the process for every pixel in the input image.

Neighborhood processing

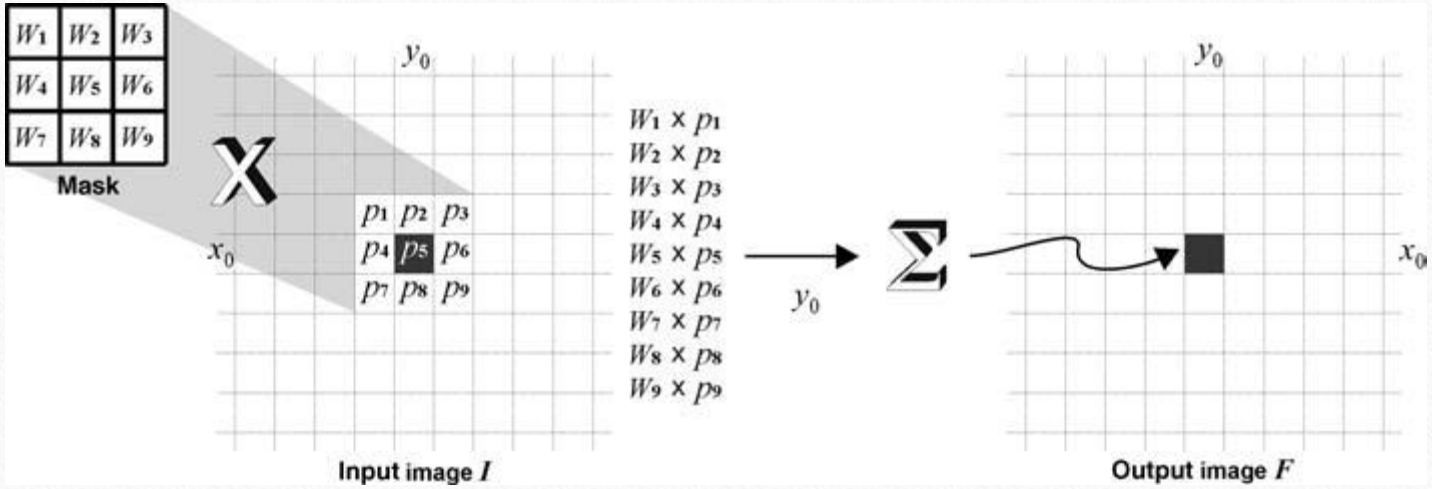
- Linear Filters: sum of products of the pixel values and mask coefficients in the pixel's neighborhood in the original image
- Nonlinear Filters: resulting output pixel is selected from an ordered (ranked) sequence of pixel values in the pixel's neighborhood in the original image

CONVOLUTION AND CORRELATION

- Convolution and correlation are the two fundamental mathematical operations involved in linear neighborhood-oriented image processing algorithms.
- The two operations differ in a very subtle way.

CONVOLUTION

- Computing for each pixel a weighted sum of the values of that pixel and its neighbors.



1D convolution

The convolution between two discrete one-dimensional arrays $A(x)$ and $B(x)$, denoted by $A * B$, is mathematically described by the equation:

$$A * B = \sum_{j=-\infty}^{\infty} A(j) \cdot B(x - j) \qquad (10.1)$$

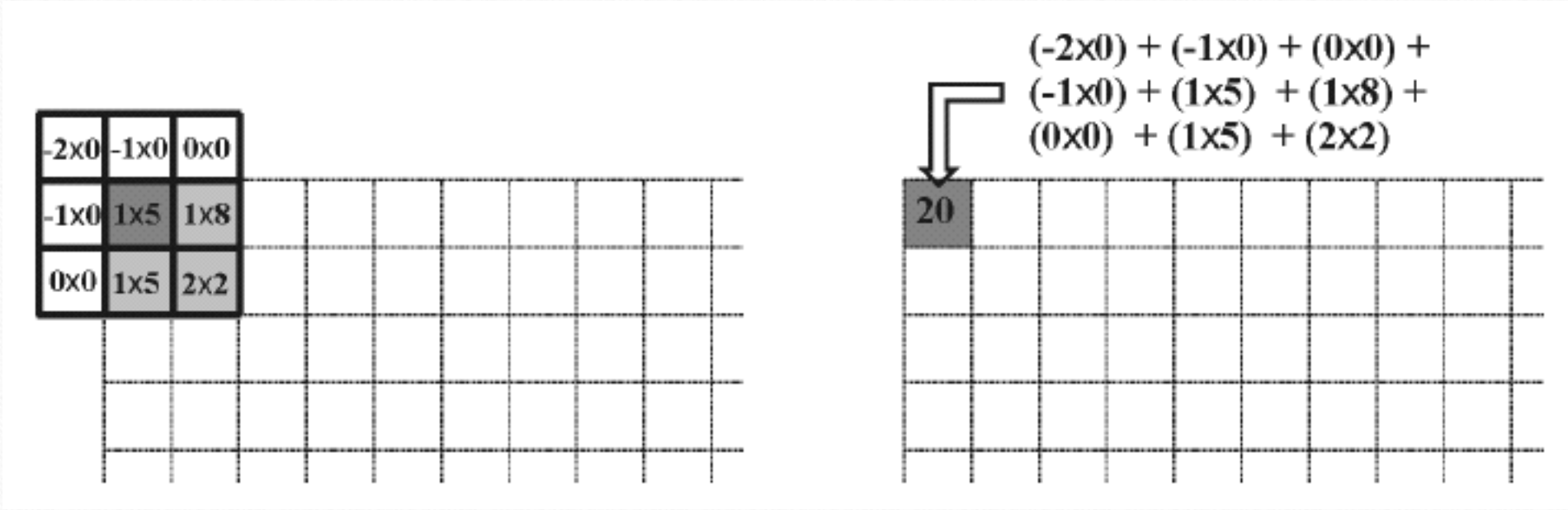
- See Example 10.1

2D convolution

$$g(x,y) = \sum_{k=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} h(j,k) \cdot f(x-j, y-k)$$

$$g(x,y) = \sum_{k=-n_2}^{n_2} \sum_{j=-m_2}^{m_2} h(j,k) \cdot f(x-j, y-k)$$

Example

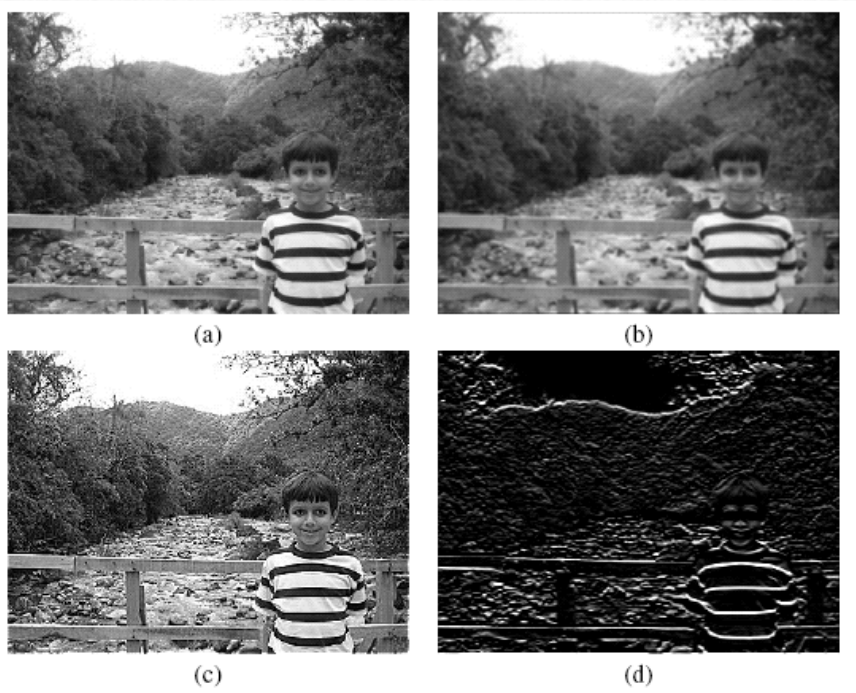

-2x0 -1x0 0x0
-1x0 1x5 1x8
0x0 1x5 2x2
(-2x0) + (-1x0) + (0x0) +
(-1x0) + (1x5) + (1x8) +
(0x0) + (1x5) + (2x2)
20

Convolution and correlation

- Convolution with different masks
- Convolution is a very versatile image processing method.
- Depending on the choice of mask coefficients, entirely different results can be obtained.

Low-pass filter	High-pass filter	Horizontal edge detection
$\begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$

Con relation



- Convolution masks

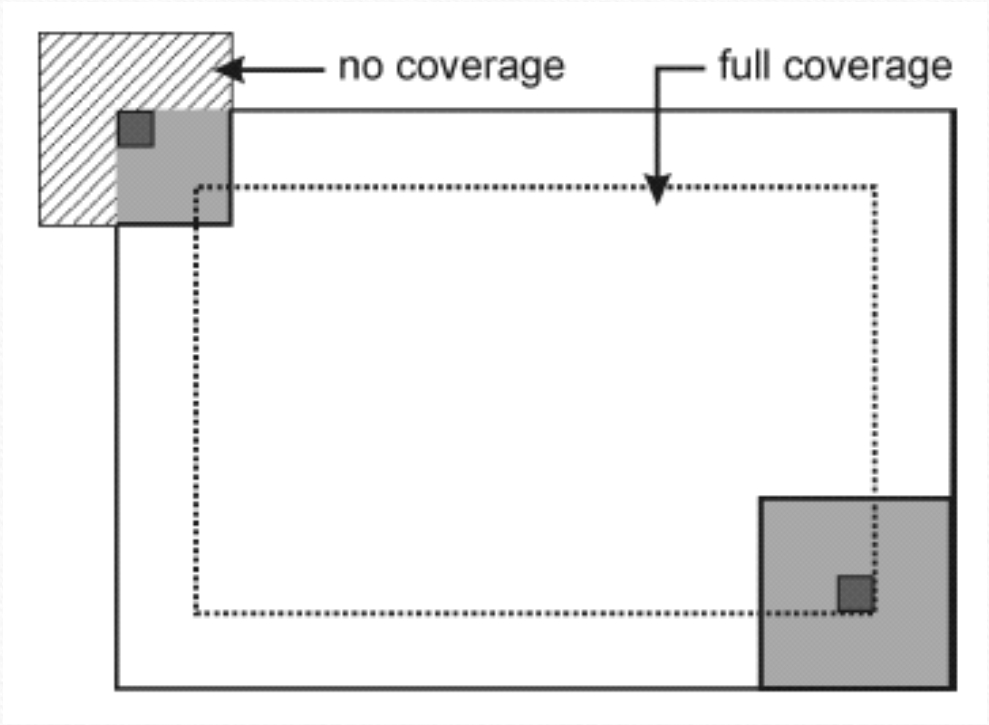
Convolution and correlation

- Correlation
  - Simply put, correlation is the same as convolution without the mirroring (flipping) of the mask before the sums-of-products are computed.
  - The difference between using correlation and convolution in 2D neighborhood processing operations is often irrelevant because many popular masks used in image processing are symmetrical around the origin.

Convolution in MATLAB

- conv2: computes the 2D convolution between two matrices. In addition to the two matrices it takes a third parameter that specifies the size of the output.
- filter2: rotates the convolution mask (which is treated as a 2D FIR filter) 180° in each direction to create a convolution kernel and then calls conv2 to perform the convolution operation.

Dealing with image borders



Dealing with image borders: options

1. Two variants of this approach:
   - Keep the pixel values that cannot be reached by the overlapping mask untouched.
   - Replace the pixel values that cannot be reached by the overlapping mask with a constant fixed value, usually zero (black).
2. Pad the input image with zeros.
3. Pad with extended values.
4. Pad with mirrored values.
5. Treat the input image as a 2D periodic function whose values repeat themselves in both horizontal and vertical directions.

- In MATLAB: check the boundary_options parameter for function **imfilter.**

Image smoothing (Low-Pass Filters)

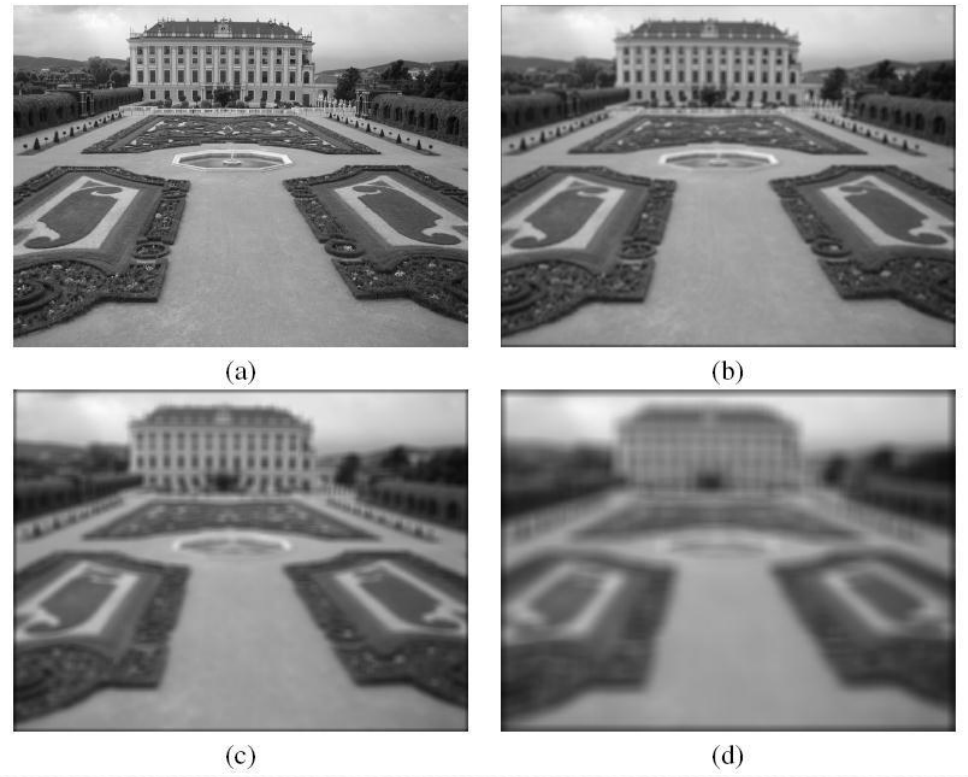
- Spatial filters whose effect on the output image is equivalent to attenuating high-frequency components (i.e., fine details in the image) and preserving low-frequency components (i.e., coarser details and homogeneous areas in the image).
- Linear LPFs can be implemented using 2D convolution masks with non-negative coefficients.
- Linear LPFs are typically used to either blur an image or reduce the amount of noise present in the image.
- In MATLAB: **imfilter and fspecial**

Mean (averaging) filter

- The simplest and most widely known spatial smoothing filter.
- It uses convolution with a mask whose coefficients have a value of 1, and divides the result by a scaling factor (the total number of elements in the mask).
- Also known as *box filter*.

$$h(x,y) = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Mean (averaging) filter: impact of mask size



Mean (averaging) filter: variations

- Modified mask coefficients, e.g.:

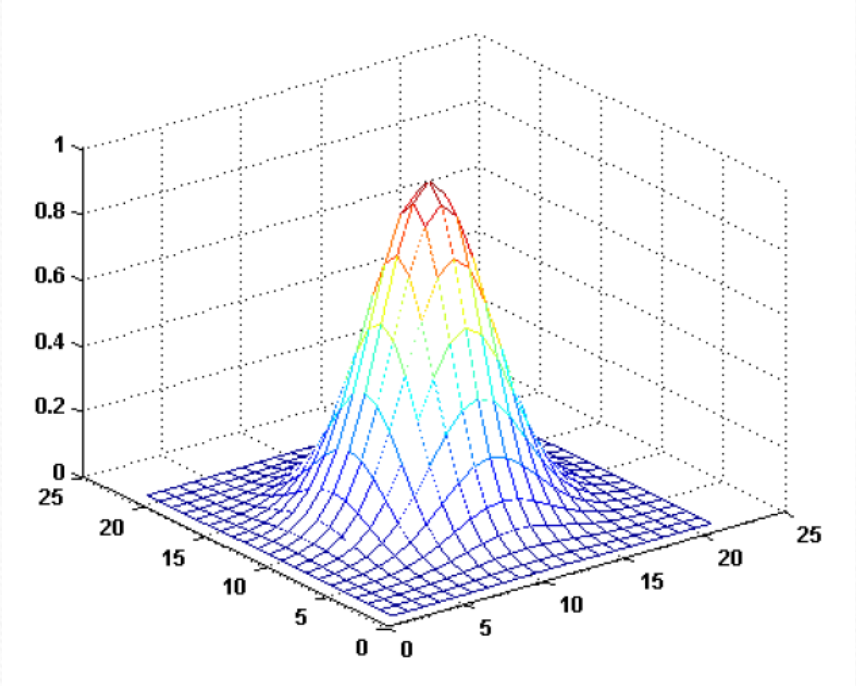
$$h(x,y) = \begin{bmatrix} 0.075 & 0.125 & 0.075 \\ 0.125 & 0.2 & 0.125 \\ 0.075 & 0.125 & 0.075 \end{bmatrix}$$

- Directional averaging
- Selective application of averaging calculation results
- Removal of outliers before calculating the average

Gaussian blur filter

- The best-known example of a LPF implemented with a non-uniform kernel.
- The mask coefficients for the Gaussian blur filter are samples from a 2D Gaussian function:

$$h(x, y) = \exp\left[\frac{-(x^2 + y^2)}{2\sigma^2}\right]$$



Gaussian blur filter

- Properties:
  - The kernel is symmetric w.r.t rotation, therefore there is no directional bias in the result.
  - The kernel is separable, which can lead to fast computational implementations.
  - The kernel's coefficients fall off to (almost) zero at the kernel's edges.
  - The Fourier Transform (FT) of a Gaussian filter is another Gaussian (this will be explained in Chapter 11).
  - The convolution of two Gaussians is another Gaussian.

Example 10.4

```
I = imread('Figure10_07_a.png');

h1 = fspecial('gaussian', [5 5], 1)
h2 = fspecial('gaussian', [13 13], 1);
h3 = fspecial('average', [13 13]);

J1 = imfilter(I, h1);
J2 = imfilter(I, h2);
J3 = imfilter(I, h3);
```

Original image

Gaussian filter, 5×5 mask, $\sigma = 1$

Mean filter, 13×13 mask

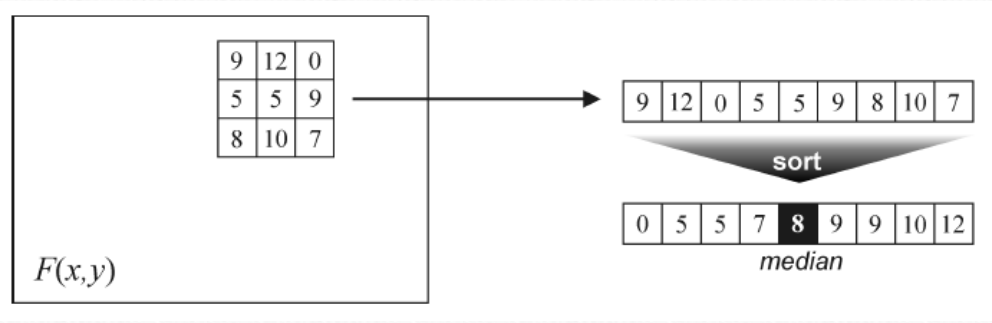
Gaussian filter, 13×13 mask, $\sigma = 1$

Median and other nonlinear filters

- Nonlinear filters also work at a neighborhood level, but do not process the pixel values using the convolution operator.
- Instead, they usually apply a ranking (sorting) function to the pixel values within the neighborhood and select a value from the sorted list.
- Sometimes called *rank filters*.
- Examples: median filter, max and min filters (Chapter 12).

Median filter

- Works by sorting the pixel values within a neighborhood, finding the median value and replacing the original pixel value with the median of that neighborhood.



Median filter

Example (salt-and-pepper noise reduction)

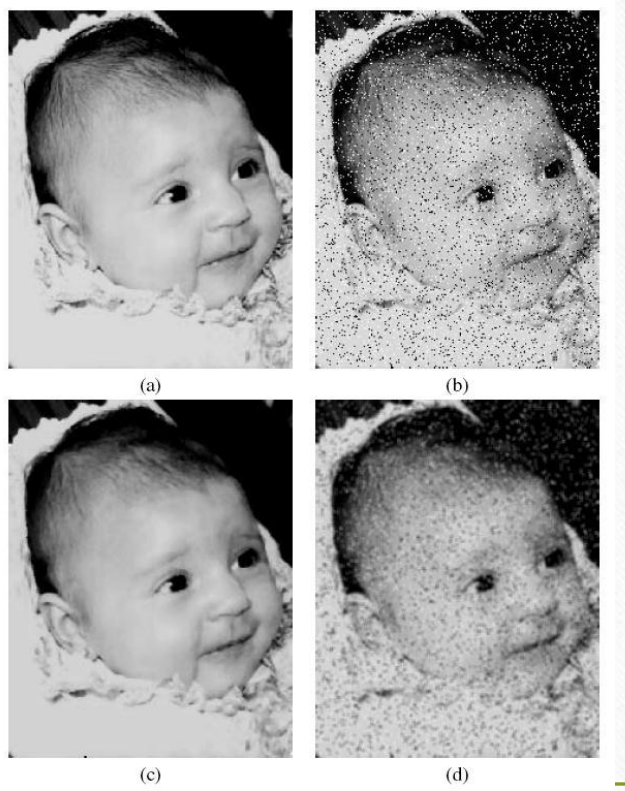


Image sharpening (High-Pass Filters)

- Spatial filters whose effect on the output image is equivalent to preserving or emphasizing its high-frequency components (e.g., fine details, points, lines, and edges), i.e. to highlight transitions in intensity within the image.
- Linear HPFs can be implemented using 2D convolution masks with positive and negative coefficients, which correspond to a digital approximation of the *Laplacian, a simple, isotropic (i.e., rotation invariant) second-order derivative that is capable of responding to intensity transitions in any direction.*

Image sharpening (HPF)

- The *Laplacian*

The Laplacian of an image $f(x, y)$ is defined as:

$$\nabla^2(x, y) = \frac{\partial^2(x, y)}{\partial x^2} + \frac{\partial^2(x, y)}{\partial y^2}$$

$$\nabla^2(x, y) = f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1) - 4f(x, y)$$

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Image sharpening (HPF)

- Composite Laplacian mask

$$g(x,y) = f(x,y) + c\left[\nabla^2(x,y)\right]$$

- For *c=1:*

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Image sharpening (HPF)



```
I = imread('coat_of_arms_before.png');
h = fspecial('laplacian', 0);
I1 = im2double(I);
J = imfilter(I1,h);
K = I1-J;
h8 = [1 1 1; 1 -8 1; 1 1 1]
K8 = I1 - imfilter(I1,h8,'replicate');
Ja = J + 0.75;
```

Directional difference filters

- Similar to the Laplacian high-frequency filter.
- Main difference: directional difference filters emphasize edges in a specific direction.
- Usually called *emboss filters*.
- Examples of masks that can be used to implement the emboss effect:

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Unsharp masking

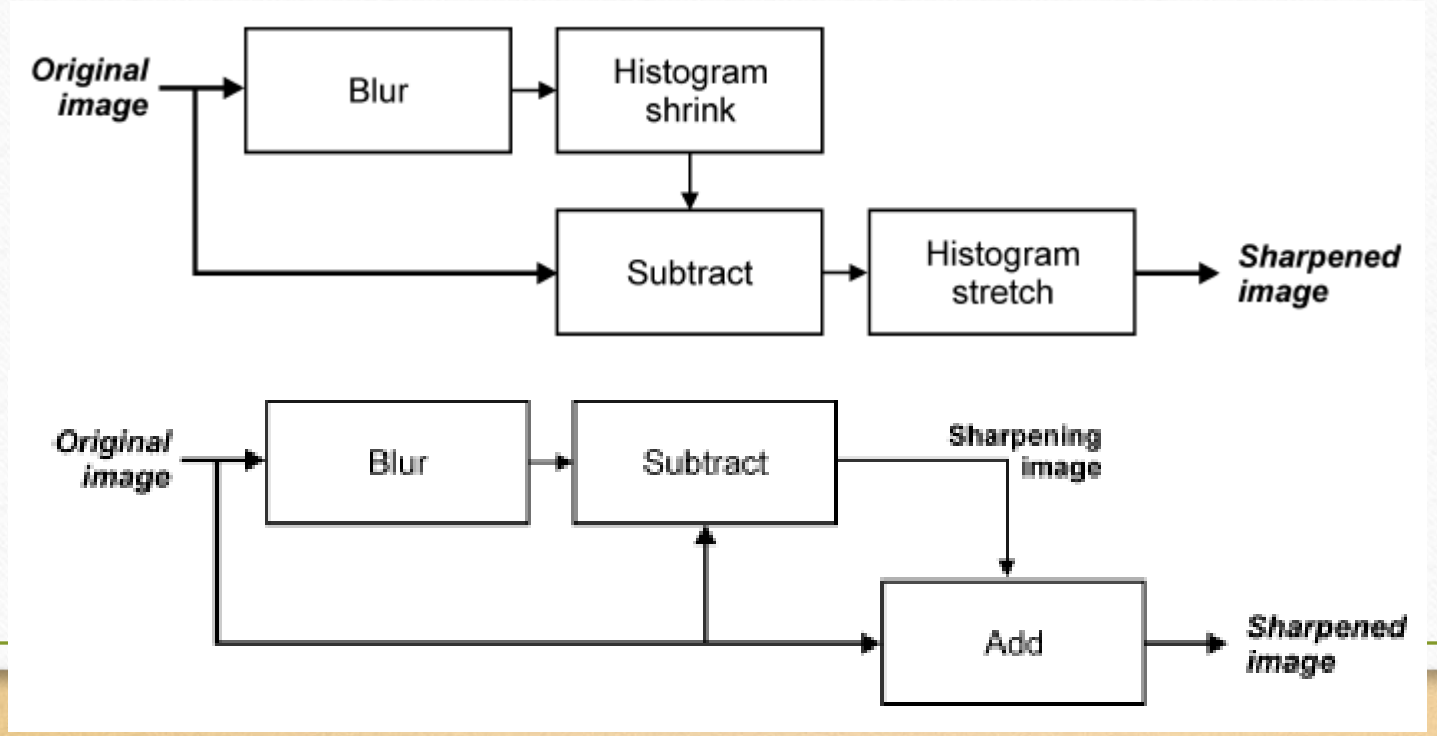
- Consists of computing the subtraction between the input image and a blurred (low-pass filtered) version of the input image.
- Rationale: to “increase the amount of high-frequency (fine) detail by reducing the importance of its low-frequency contents”.

Unsharp masking

- Variants (see Tutorial 10.3):

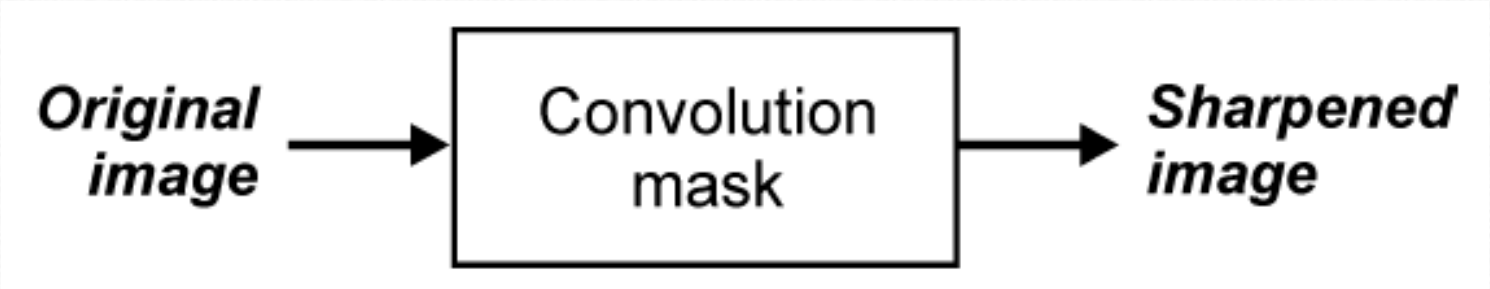
(1)

(2)



Unsharp masking

- Variants (see Tutorial 10.3):
- (3)



High-boost filtering

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & c & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- c (c > 8) is a coefficient ("amplification factor")

that controls how much weight is given to the original image and the high-pass filtered version of that image.

- For c=8, the results would be equivalent to those seen earlier for the conventional isotropic Laplacian mask.
- Greater values of c will cause significantly less sharpening.

ROI Processing

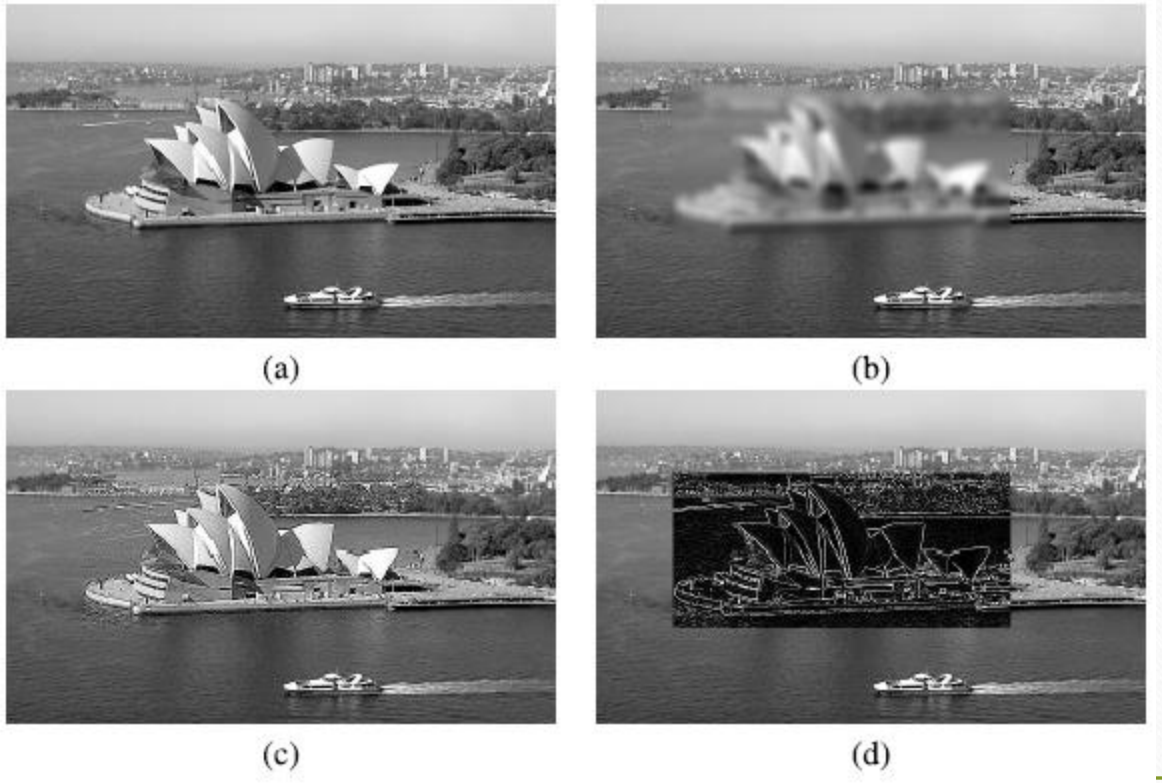
- Filtering operations are sometimes performed only in a small part of an image, known as a *region of interest (ROI)*, which can be specified by defining a (usually binary) mask.
- Image masking is the process of extracting such a subimage (or ROI) from a larger image for further processing.
- In MATLAB: A combination of two functions: **roipoly (see Tutorial 6.2) and roifilt2**

ROI Processing

- Example 10.6:

```
I = imread('Figure10_11_a.png');
r = [90 254 254 90];
c = [84 84 447 447];
BW = roipoly(I,c,r);
h = fspecial('gaussian', [15 15], 5);
J = roifilt2(h,I,BW);
h8 = [1 1 1; 1 -8 1; 1 1 1]
K = roifilt2(h8,I,BW);
h4 = [0 -1 0; -1 5 -1; 0 -1 0]
L = roifilt2(h4,I,BW);
```

ROI Processing



Combining spatial enhancement methods

- When faced with a practical image processing problem, a question arises: **Which techniques should I use and in which sequence?**
- There is no universal answer to this question.
- Most image processing solutions are problem-specific and involve the application of several algorithms -- in a meaningful sequence -- to achieve the desired goal.
- The choice of algorithms and fine-tuning of their parameters is a trial-and-error process.
- Using the knowledge acquired so far you should be able to implement, configure, fine-tune, and combine image processing algorithms for a wide variety of real-world problems.

Hands-on

- Tutorial 10.1: Convolution and correlation (page 223)
- Tutorial 10.2: Smoothing filters in the spatial domain (page 225)
- Tutorial 10.3: Sharpening filters in the spatial domain (page 228)